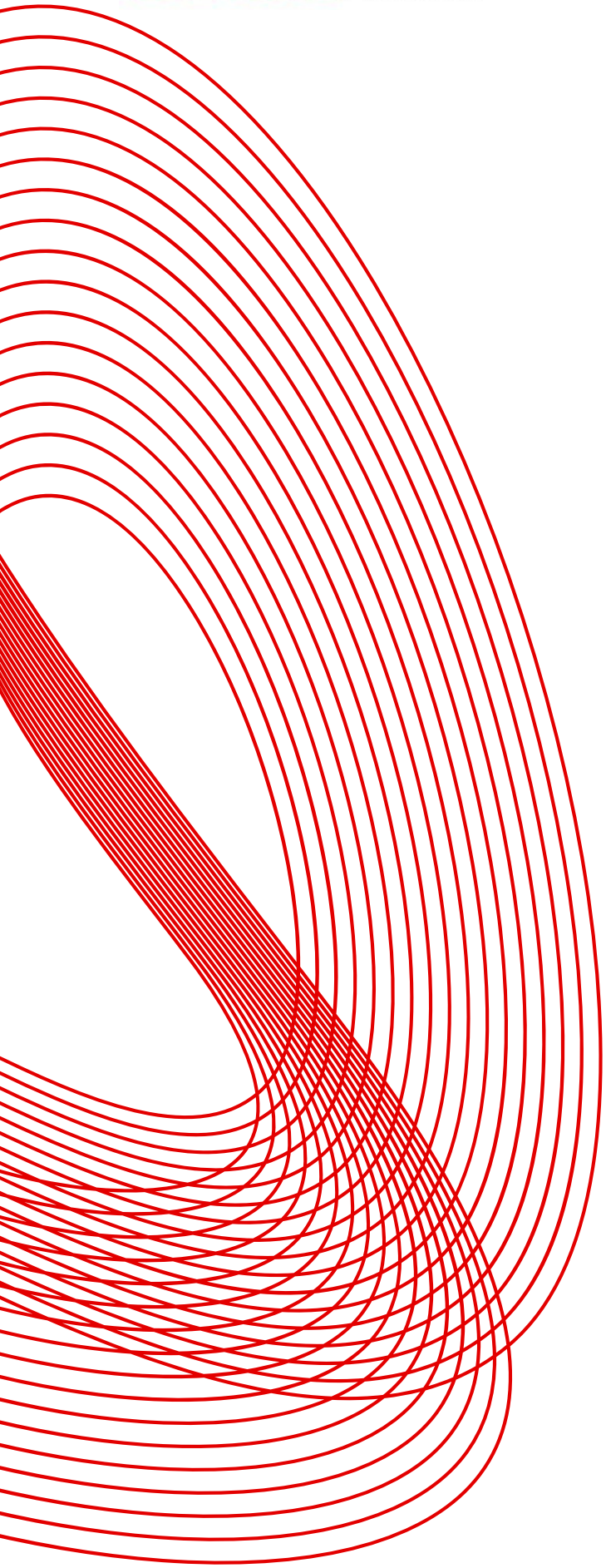
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES STRASBOURG

S9



S-IoT Project Report :

Drone swarm

Written by :

Emilien PEYON

Noé RAMDANI

Class of 2022-2027

Specifications

Objective

The purpose of this project is to fly a swarm of UAVs (2 to 3 UAVs) and try to lift off a payload using a net.

Functions

- UWB precise location system
- PID Controller for drone stabilization
- UWB Communication for high frequency refreshing data
- Go back and forth keeping the drone formation (following a leader ?)

Deliverables

- Report with all progress, advancements and explanation of technical knowledge
- Functional UWB code
- Methodology and results of calibration of UWB sensors
- Bibliographical Studies
- Video of drone demo
- Functional PID Corrector
- Functional ESP32 code implementation using Multi-threading

Bibliographical Studies

0.1 UWB

0.1.1 Definitions

Explain what is UWB

0.1.2 Algorithm

Explain that we found and test different Algorithm

Two way ranging (TWR)

The Two Way Ranging (TWR) algorithm is the easiest and the fastest to implement in a UWB system. In order to explain this algorithm, we will use 2 modules called UWB_1 and UWB_2.

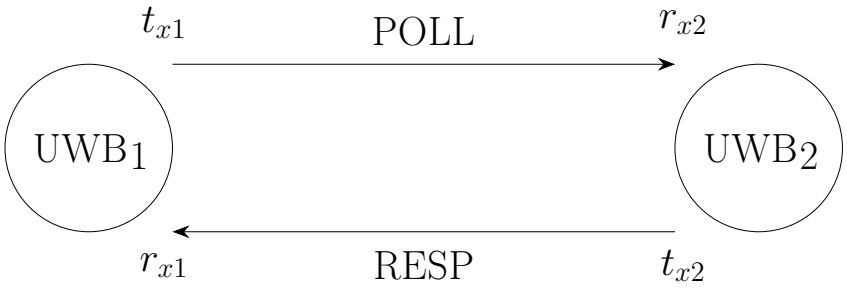


Figure 1: TWR explanation diagram

The algorithm works in 4 steps :

1. UWB_1 sends a **POLL** to UWB_2. The time of emission t_{x1} is memorized by UWB_1.
2. UWB_2 receives the **POLL** and memorizes the time of arrival r_{x2}.
3. UWB_2 processes the **POLL**, and precisely plans the time of emission t_{x2} for the **RESP**. It contains (t_{x2}, r_{x2}).
4. UWB_1 receives the **RESP** and memorizes the r_{x1}.

With all those parameters, we can deduce those variables :

- Total time : $t_{tot} = r_{x1} - t_{x1}$
- Processing time of UWB_2 : $t_{proc} = r_{x2} - t_{x2}$
- Travel time : $t_{travel} = t_{tot} - t_{proc}$

The signal travels back and forth so it did twice the distance (d), and travel speed of a radio signal is equivalent to light speed (c).
So, we can deduce this equation :

$$\boxed{\frac{t_{travel} \times c}{2} = \frac{(t_{tot} - t_{proc}) \times c}{2} = \frac{((r_{x1} - t_{x1}) - (r_{x2} - t_{x2})) \times c}{2}} \quad (1)$$

We can also determine the induced error of this algorithm.
After calculation (describe in calculation part at page 7), we obtain this equation:

$$\epsilon \approx \frac{t_{proc}(e_a - e_b)}{2} \tag{2}$$

As we can observe, the Time of processing is directly link to the error. The t_{proc} is around 3,000,000 nanoseconds so, it's inducing a high error.

Double Sided Two way ranging (DS-TWR)

The DS-TWR has for purpose to reduce the error introduce by the t_{proc}. This algorithm also use UWB_1 and UWB_2.

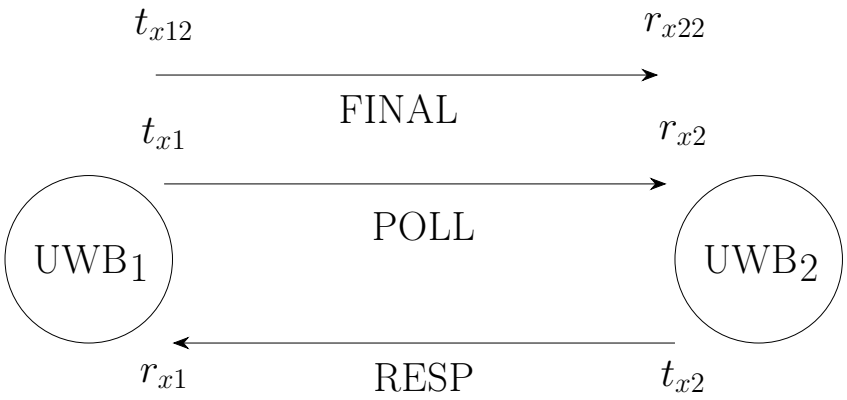


Figure 2: DS-TWR explanation diagram

0.2 Data treatment

0.2.1 Polynomial Filter

0.2.2 Extended Kalman Filter for relative localisation

In the context of navigating systems in noisy environments with sensors, the Kalman filter is the most relevant algorithm. As Welch and Bishop [1] point out, the Kalman filter is notable for its ability to "support estimations of past, present, and even future states [...] even when the precise nature of the modeled system is unknown.". In our study case, we deal with a drone swarm communicating using the UWB communication protocol, which gives us the relative distance between each pair of drones by using the time of flight of a radio wave. This measure is non-linear (Euclidian distance calculated with a square root) which implies that an extended version of the filter should be used : it resorts to a typical technique of linearization around the current state using a Jacobian matrix. We are also considering an anchor-free system which enhances the challenge of this project. In this section, we will first define the theory of the EKF (Extended Kalman Filter) and then, we will study its application to our UWB drone swarm system.

Theory of the Kalman filter

Origin of the theory In his pioneering article of 1960 [2], Rudolf Kalman describes state estimation as a recursive process in which at each time step, the current estimation and the uncertainty are computed using only the past estimate and the newly available measure. When assuming a linear dynamical system and Gaussian noise, the Kalman filter minimizes the mean-square estimation error : in other words, the best possible estimate given the available measurements. Even when these assumptions do not hold exactly, the filter remains the best linear estimator achievable (known as the Best Linear Unbiased Estimator, or BLUE). This optimality is what established the Kalman filter as the standard approach for state estimation in noisy dynamic systems. While Kalman filter's optimality holds for linear dynamical systems, we have in our case, as mentioned earlier, a non-linearity with the use of the Euclidian distance. This is why we require an extended version of the filter that can handle the non-linearity, even at the cost of losing the strict optimality.

Formalism of the discrete Kalman Filter Let's note $x_k \in \mathbb{R}_n$ the current state of a discrete system governed by the following linear equation :

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$

In which we have :

- F_k is the state transition matrix, linking the previous state x_{k-1} to the current state x_k
- B_k is the control-input matrix, linking the control vector u_k to the current state x_k
- w_k is the process noise which is assumed to be Gaussian and centered, with a covariance matrix Q_k

At each time k, the measurement z_k is written as follows :

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

With :

- H_k is the observation matrix
- w_k is the observation noise which is assumed to be Gaussian and centered, with a covariance matrix R_k

We can divide the Kalman filter process into 2 phases : the "prediction" phase and the "update" phase. The first one does an estimate of the state at the next time step by using the estimate of the current time step and the error covariance. The second phase can be thought of as a feedback : it includes a new measurement in the current state to improve its estimate.

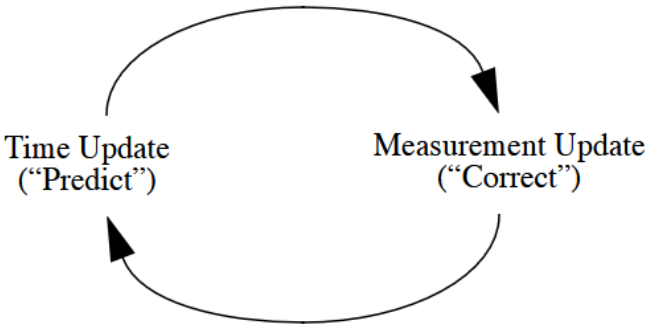


Figure 3: Ongoing discrete Kalman filter cycle (Welch and Bishop (2006, p. 5) [1])

Extended Kalman filter

As mentioned earlier, the extended Kalman filter is used to deal with a non-linear state process defined as diffentiable function f. To do so, it essentially linearizes the nonlinear function around the current estimate by computing the Jacobian matrices of the nonlinear functions f and h in the following equations (with the same notations as before) :

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)$$
$$\mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k)$$

Caculations

TWR Error calculation :

Let's set:

- True time of flight : T_f
- True time of process : t_{proc}
- Clock drift errors : e_a and e_b

The measured times are:

$$R_a = (2T_f + t_{proc})(1 + e_a) \text{ and } t_{proc_b} = t_{proc}(1 + e_b) \tag{3}$$

The calculated time of flight is:

$$\hat{T}_f = \frac{R_a - t_{proc_b}}{2} \tag{4}$$

By replacing R_a and t_{proc_b}, we obtain :

$$\hat{T}_f = \frac{2T_f(1 + e_a) + t_{proc}(1 + e_a) - t_{proc}(1 + e_b)}{2} \tag{5}$$

By simplifying, we obtain :

$$\hat{T}_f = T_f(1 + e_a) + \frac{t_{proc}(e_a - e_b)}{2} \tag{6}$$

The error is $\hat{T}_f - T_f$, so:

$$Error = T_f \times e_a + \frac{t_{proc}(e_a - e_b)}{2} \tag{7}$$

The time of flight is about 33 nanoseconds, however D is around $3,000,000$.
So we can ignore $T_f \times e_a$ cause $T_f \ll t_{proc}$, and we obtain :

$$Error \approx \frac{t_{proc}(e_a - e_b)}{2} \tag{8}$$

DS-TWR Error calculation :

References

[1] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," University of North Carolina at Chapel Hill, Tech. Rep. TR 95-041, 2006.

[2] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," en, *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960, ISSN: 0021-9223. DOI: 10.1115/1.3662552. [Online]. Available: https://asmedigitalcollection.asme.org/fluidsengineering/article/82/1/35/397706/A-New-Approach-to-Linear-Filtering-and-Prediction (visited on 10/05/2026).